

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



REGISTRO DE USO DE IA - EJERCICIO 2



Ezequiel Pino

Registro de Uso de Inteligencia Artificial -

Ejercicio 2 - Queries

Para la ejecución del desafío, trabaje exactamente igual que en el ejercicio 1, utilice múltiples modelos, en distintas fases, según pruebas previas y benchmarks estudiados, para poder seleccionar el mejor modelo, para cada tarea.

La idea, nuevamente, era utilizar criterios eficientes, para poder expresar al máximo el potencial de cada modelo.

A diferencia del Ejercicio 1, en este caso el objetivo principal consistía en analizar un repositorio de Enterprise Architect y construir un conjunto acotado de consultas SQL. Por este motivo mantuve la metodología de Desarrollo Guiado por Especificaciones —SDD—, pero adapté la profundidad de cada fase al alcance del ejercicio. Realice la parte opcional de Prolaborate, manualmente.

Aclaración: En la columna de “Estrategia-Prompt”, y en la columna de “Evidencia”, decidí colocar una referencia en la celda para aquellas peticiones largas. Utilizando la referencia, se puede buscar el prompt debajo del recuadro, es una decisión simplemente estética y de orden.

Fase de Desarrollo

Al igual que en el ejercicio 1, utilizamos la metodología SDD, dejo adjuntado nuevamente un cuadro a modo de explicación de cada fase:

Fila del TUI	UI / Abstracción anterior	Función real
gentle-orchestrator	Coordinador (no es fase)	Agente coordinador del perfil: entiende la intención, decide cuándo entrar a SDD, delega a los subagentes de cada fase y sostiene el hilo de la sesión completa.
sdd-init	Inicialización	Detecta stack, scripts, convenciones, pruebas y estado mínimo del proyecto.
sdd-explore	Exploración	Investiga flujos, dependencias, archivos clave, riesgos y áreas impactadas.
sdd-propose	Propuesta / PRD operativo	Convierte la intención en alternativa recomendada, alcance, riesgos y criterios de decisión.
sdd-spec	Especificación OpenSpec	/ Formaliza requisitos, contratos, criterios de aceptación y comportamiento esperado.
sdd-design	Diseño técnico	Define arquitectura, cambios por capas, estrategia de datos, APIs, UI y pruebas.
sdd-tasks	Plan de tareas	Divide el diseño en pasos ejecutables, ordenados y verificables.
sdd-apply	Implementación	Aplica cambios en código, archivos, configuraciones, tests y scripts.
sdd-verify	Review / Audit	Valida diff, pruebas, riesgos, regresiones, seguridad y cumplimiento de spec.
sdd-archive	Cierre Commit-ready	/ Resume artefactos, deja bitácora, notas de commit y estado final.

En lugar de desarrollar las consultas directamente a partir de una conversación libre con un modelo, el trabajo se dividió en etapas con responsabilidades concretas. Cada fase recibió los artefactos generados por las anteriores y produjo información que debía ser revisada antes de continuar.

A continuación, nuevamente comparto los enlaces a repositorios de Github de este Ecosistema:

- Repositorio de Gentle-AI: <https://github.com/Gentleman-Programming/gentle-ai>
- Repositorio de ENGRAM: <https://github.com/Gentleman-Programming/engram>

No quiero dejar de destacar la siguiente frase:

Trabajando de esta forma, convertimos autonomía cruda de modelos, en trabajo de ingeniería eficiente y controlado.

Utilizamos OpenCode en su forma TUI para desplegar Gentle AI, pero esta vez en el entorno de Visual Studio Code, ya que íbamos a trabajar sobre SQL, buscaba un entorno mas "liviano", ya que no era necesario ejecutarlo dentro de Visual Studio.

```

+ Thought: Delegating tasks - 460ms
Delegando APPLY Q2 a sdd-apply...
✓ Sdd-Apply Task - Document Q1 and validate Q2
  ↳ 45 toolcalls · 9m 5s

ctrl+x down view subagents

+ Thought: Settling attempt with multi-file hashing - 2.6s

$ $paths = @('openspec/changes/ea-governance-queries-exercise-2/tasks.md',
'Exercise_2_queries/queries/Pino_Exercise_2_EA_Governance_Queries.sql',
'Exercise_2_queries/docs/Pino_Exercise_2_Technical_Explainer.md', 'Exercise_2_queries/
docs/Pino_Exercise_2_AI_Usage_Log.md', 'Exercise_2_queries/evidence/
Pino_Exercise_2_Evidence_Index.md', 'Exercise_2_queries/evidence/sqlite/
Pino_Exercise_2_SQLite_Diagnostics.md'); $combined = ($paths | foreach-object { (get-
FileHash -Algorithm SHA256 -LiteralPath $_).Hash.ToLowerInvariant() }) -join "; $bytes
= [Text.Encoding]::UTF8.GetBytes($combined); $hash = [System.Security.Cryptography.
SHA256]::HashData($bytes); $evidenceRevision = 'sha256:' + ([Convert]::ToHexString($hash
.ToLowerInvariant())); gentle-ai sdd-attempt settle --cwd "C:\Proagile\Addin" --change
"ea-governance-queries-exercise-2" --token "sha256:
0a66dc1e117e3c742489745b15858a01c0e807e42a1f240915a0922a2a" --request-id "Q2-Q2-
apply-settle-20260813-01" --outcome passed --evidence-revision $evidenceRevision --
diagnostics "documented operator-confirmed Q1 EA pass; Q2 primary SQL and SQLite read-only
diagnostics passed; stopped before required HUMAN GATE Q2." --harness-disposition reused
--cleanup-evidence "no cleanup required; no QEA writes, screenshots, staging, commits,
or protected-area changes." --process-evidence "Q1 EA evidence confirmed by operator; Q2
SQLite URI mode-ro passed; Q2 EA intentionally deferred to HUMAN GATE Q2."; git diff --

Gentle-Orchestrator · GPT-5.6 Terra openAI
C:\Proagile\Addin 173.4K (35%) ctrl+lp commands

```

Para este ejercicio se incorporó además una condición especialmente importante: Enterprise Architect debía actuar como superficie primaria de validación, mientras que SQLite se utilizó únicamente como herramienta diagnóstica secundaria y siempre en modo read-only. Esto permitió contrastar los resultados antes de cada validación manual sin modificar el repositorio de trabajo.

Selección de Modelos

Para este ejercicio mantuve el mismo criterio utilizado en el ejercicio anterior: asignar cada fase del proceso SDD al modelo que consideraba más conveniente según el tipo de tarea, buscando equilibrar calidad de razonamiento, consumo de tokens y preservación de la ventana de contexto. Al tratarse de un ejercicio centrado principalmente en análisis del repositorio de Enterprise Architect, definición de consultas SQL y validación documental, algunas etapas podían resolverse con modelos más livianos, mientras que otras requerían mayor capacidad de razonamiento técnico.

Perfiles SDD para Gentle-AI + OpenCode:

https://docs.google.com/document/d/15_b_dOgp6Wf5PI6RrZ6e0QGRBMXE2xnw/edit?usp=sharing&ouid=103625650383831231991&rtpof=true&sd=true

A continuación, adjunto la tabla de registro de uso que usamos para algunas fases.

Cabe recalcar que la documentación nombrada en la parte de evidencia, se encuentra en la ruta del proyecto::

\\openspec\\changes\\archive\\ea-governance-queries-exercise-2

ID	Objetivo	Herramienta	Modelo	Estrategia - Prompt	Evidencia	Resultado
QRY-001	Inicializar formalmente el Ejercicio 2 dentro del flujo SDD, delimitar el workspace y establecer las restricciones de trabajo antes de comenzar el análisis. Fase Init.	OPENCODE / Gentle-AI	GPT-5.6 Luna	Prompt: QRY-001	Inicialización del change ea-governance-queries-exercise-2 y preparación de su estructura OpenSpec.	Se estableció formalmente el alcance del Ejercicio 2, manteniendo el repositorio QEA en modo read-only y preservando los artefactos del Ejercicio 1. El entorno quedó preparado para iniciar Explore.
QRY-002	Explorar el repositorio QEA y determinar los hechos técnicos necesarios antes de definir una solución: criterio de aplicación, tagged values, DB28, dependencias, duplicados y compatibilidad con EA SQL Search. Fase Explore.	OPENCODE / Gentle-AI	MiniMax M3	Prompt: QRY-002	Creación de exploration.md.	La exploración estableció una base técnica reproducible para el ejercicio: identificó el universo de aplicaciones, la ubicación de Categoría y Vigencia, la identidad de Base de Datos 28 y la semántica de las dependencias. La fase quedó aprobada para avanzar a Proposal.

QRY-003	Definir formalmente el alcance obligatorio, estrategia de solución, restricciones y criterios de validación sin implementar todavía las consultas. Fase Proposal.	OPENCODE / Gentle-AI	GPT-5.6 Terra	Prompt: QRY-003	Creación de proposal.md; utilización de exploration.md como base aprobada.	Se definió una solución SELECT/read-only, con EA SQL Search como superficie primaria de validación, SQLite mode=ro como diagnóstico secundario y Prolaborate como seguimiento opcional posterior a la validación obligatoria.
QRY-004	Convertir Explore, Proposal y la consigna en requisitos y escenarios verificables que definieran exactamente el comportamiento esperado antes de diseñar el SQL final. Fase Spec.	OPENCODE / Gentle-AI	Kimi K2.7 Code	Prompt: QRY-004	Creación del conjunto de Specs bajo specs/, separando los requisitos de Categoría, Vigencia e Impacto sobre Base de Datos 28.	Se obtuvieron tres capability Specs verificables, cubriendo el universo de aplicaciones, tagged values, semántica de DB28, read-only, evidencia EA, trazabilidad y restricciones del ejercicio. Las Specs quedaron aprobadas antes de Design.
QRY-005	Diseñar la solución técnica definitiva: descomposición del query set, estrategia de unicidad, compatibilidad EA, evidencia y gates de validación. Fase Design.	OPENCODE / Gentle-AI	GPT-5.6 Sol	Prompt: QRY-005	Creación de design.md; utilización de proposal.md y las Specs aprobadas.	Se definieron cinco SELECT finales : Q1-C, Q1-L, Q2-G, Q3-C y Q3-L. También se establecieron DISTINCT/COUNT(DISTINCT), alias de navegación EA, identificación semántica de DB28, validación EA primaria, SQLite secundaria y los HUMAN GATES incrementales.

QRY-006	Transformar el diseño aprobado en un plan incremental, ejecutable y verificable, separando claramente las tareas automatizables de las validaciones humanas obligatorias. Fase Tasks.	OPENCODE / Gentle-AI	Kimi K2.7 Code	Prompt: QRY-006	Creación de tasks.md; utilización de exploration.md, proposal.md, Specs y design.md como fuentes aprobadas.	Se generó un plan incremental organizado en work units: scaffolding, Q1 → Gate Q1, Q2 → Gate Q2, Q3 → Gate Q3, documentación final, Prolaborate opcional y pre-verify. La implementación debía comenzar únicamente por Q1.
----------------	---	----------------------	----------------	------------------------	---	--

Prompts

QRY-001

Vamos a iniciar formalmente el Exercise 2 del desafío técnico Proagile 2026 en este workspace.

ROOT DEL WORKSPACE: C:\Proagile\Addin

CAMBIO SDD NUEVO: ea-governance-queries-exercise-2

ÁREA EXCLUSIVA DE IMPLEMENTACIÓN/ENTREGA DEL EXERCISE 2:

C:\Proagile\Addin\Exercise_2_queries

REPOSITORIO EA DE TRABAJO: C:\Proagile\Data\Repositorio Pasantias.qea

IMPORTANTE SOBRE EL WORKSPACE EXISTENTE:

- `Addino/` contiene el Exercise 1 ya implementado, verificado, archivado y funcional en Enterprise Architect.

- NO modificar código, configuración, documentación ni binarios de `Addino/`.

- NO reabrir el cambio archivado del Exercise 1.

- El historial del Exercise 1 está preservado bajo `openspec/changes/archive/`.

- `openspec/specs/ea-metadata-review/` es una spec canónica histórica de E1 y no pertenece al nuevo alcance.

- El store OpenSpec de este workspace está en `C:\Proagile\Addin\openspec`.

OBJETIVO DEL EXERCISE 2:

Construir queries SQL sobre el repositorio de Enterprise Architect para responder, como mínimo:

1. Cuántos elementos de tipo aplicación tienen el tagged value `Categoria` con valor `ORO` y cuáles son.

2. Cuántas aplicaciones existen en cada estado de vigencia `Vigente` y `Deprecado`, contemplando valores vacíos/no informados si corresponde.

3. Qué aplicaciones se verían afectadas si se da de baja `Base de Datos 28`, identificando dependencias cuyo origen sea una aplicación y cuyo destino sea `Base de Datos 28`.

REQUISITOS DE LA CONSIGNA:

- Identificar explícitamente las tablas principales del repositorio EA utilizadas para elementos, relaciones y tagged values.
- Las queries finales deberán ser comprensibles, reutilizables y verificables.
- Deberán ejecutarse en el motor SQL/Search de Enterprise Architect o mecanismo equivalente permitido.
- Los resultados deberán validarse manualmente contra elementos visibles del modelo.
- La query de Categoría debe documentar qué campo/tagged value determina la categoría.
- La query de vigencia debe contemplar valores vacíos/no informados si realmente existen.
- La query de impacto debe respetar estrictamente la dirección origen aplicación -> destino Base de Datos 28.
- Los entregables posteriores incluirán archivo SQL, explicación técnica de tablas/joins/filtros/supuestos, evidencia EA y Registro de Uso de IA.
- Existe un punto extra posterior para llevar una query validada a un gráfico/dashboard Prolaborate. NO implementar todavía Prolaborate.

FUENTES DISPONIBLES/RELEVANTES:

- Consigna oficial Proagile.
- `Repositorio Pasantias.qea`.
- `InsideEA.pdf`, especialmente la documentación de `t_object`, `t_objectproperties` y `t_connector`.
- Enterprise Architect Object Model 17.1.
- Enterprise Architect SDK.
- Documentación Prolaborate V5 `Build Donut Charts with Live Information in Prolaborate`, solamente para la fase opcional posterior.

REGLAS PARA ESTA SESIÓN:

1. Recupera mediante ENGRAM cualquier contexto útil del proyecto anterior, especialmente organización del workspace, política de evidencia y proceso SDD, pero NO reutilices decisiones funcionales de Exercise 1 para inventar decisiones de Exercise 2.
2. Consulta el estado SDD nativo antes de crear o modificar artefactos.
3. Inicializa/onboarda el nuevo cambio según el mecanismo nativo disponible.
4. Ejecuta únicamente las fases necesarias de INIT/ONBOARD y EXPLORE.
5. Usa `sdd-init` con GPT-5.6 Luna, `sdd-onboard` con MiniMax M3 cuando corresponda y `sdd-explore` con MiniMax M3.
6. NO ejecute todavía `sdd-propose`, `sdd-spec`, `sdd-design`, `sdd-tasks`, `sdd-apply`, `sdd-verify` ni `sdd-archive`.
7. NO escribas todavía las tres queries finales.
8. NO modifiques el repositorio QEA. Toda inspección de `Repositorio Pasantias.qea` debe ser estrictamente READ-ONLY.
9. Puedes inspeccionar el esquema y datos del QEA mediante SQLite/read-only tooling si está disponible.
10. No asumir que `Aplicación` equivale simplemente a un `Object_Type` concreto: demuestra cómo se identifica realmente en este repositorio mediante tipo, stereotype, paquete u otros metadatos.
11. No asumir que `Vigente/Deprecado` corresponde a `t\_object.Status`: inspecciona los datos reales y demuestra dónde está modelada la vigencia.

12. Para `Categoria`, verifica nombre exacto del tagged value, tabla, columna, relación con el elemento y valores existentes.
 13. Para `Base de Datos 28`, identifica de forma inequívoca el elemento, su tipo/stereotype y cómo se materializan las dependencias.
 14. Para conectores, verifica cuál columna representa origen/source y cuál destino/target en ESTE repositorio y contrástalo con la documentación.
 15. Identifica posibles duplicados de tagged values, NULL, valores vacíos o cualquier condición que pueda alterar COUNTs o producir filas duplicadas.
 16. No utilices UPDATE, INSERT, DELETE ni cualquier SQL mutante.
 17. Mantén separado el optional de Prolaborate del alcance obligatorio.
 18. No alteres RDD ni review mode durante esta exploración salvo que el runtime nativo lo exija explícitamente; informa cualquier bloqueo antes de tomar decisiones destructivas.
 19. No hagas commits Git automáticamente.
- Detente al finalizar Explore y espera aprobación humana antes de Proposal.

QRY-002

El bootstrap de Exercise 2 ya fue revisado y aprobado por el operador.
`openspec/config.yaml` representa correctamente el workspace multi-exercise y el área activa SQL/read-only.

Continúa ahora con:

1. creación nativa del cambio SDD `ea-governance-queries-exercise-2` mediante `sdd-new`;
2. únicamente la fase `sdd-explore` usando MiniMax M3.

NO continúes a Proposal, Spec, Design, Tasks, Apply, Verify ni Archive.

ROOT:C:\Proagile\Addin

ÁREA ACTIVA: C:\Proagile\Addin\Exercise_2_queries

REPOSITORIO EA: C:\Proagile\Data\Repositorio Pasantias.qea

El repositorio QEA debe inspeccionarse exclusivamente en modo read-only.

HALLAZGOS PRELIMINARES DE UNA INSPECCIÓN READ-ONLY INDEPENDIENTE:

Estos datos son evidencia inicial para acelerar Explore. NO deben aceptarse ciegamente: debes verificarlos directamente contra el QEA, documentar cómo los reprodujiste y señalar cualquier discrepancia.

1. El archivo QEA parece ser SQLite y contiene al menos:

- t_object
- t_objectproperties
- t_connector
- t_package
- t_xref
- t_stereotypes

2. La inspección encontró 30 elementos que parecen representar aplicaciones mediante:

- Object_Type = 'Class'
- Stereotype = 'ArchiMate_ApplicationComponent'

La pertenencia al universo "Aplicación" parece depender principalmente del stereotype, ya que existen otros elementos Object_Type='Class'. Verificalo.

3. Categoría parece ser un tagged value:

- t_objectproperties.Property = 'Categoría'
- t_objectproperties.Value contiene el valor.
- join preliminar: t_object.Object_ID = t_objectproperties.Object_ID.

4. Para las aplicaciones identificadas preliminarmente, Categoría produjo:

- ORO: 8
- PLATA: 8
- BRONCE: 4
- N/A: 10

Verifica todos los conteos y detecta NULL/vacíos/duplicados.

5. Las aplicaciones ORO encontradas preliminarmente fueron:

- Aplicación 1
- Aplicación 2
- Aplicación 3
- Aplicación 4
- Aplicación 6
- Aplicación 8
- Aplicación 20
- Aplicación 29

No uses esta lista como resultado final sin reproducirla mediante consulta.

6. El estado de vigencia NO parece corresponder a t_object.Status.

Se encontró como tagged value:

- t_objectproperties.Property = 'Vigencia'

Para las aplicaciones:

- Vigente: 16
- Deprecado: 14
- vacíos/N/A: aparentemente 0 dentro del universo de aplicaciones.

Existen valores N/A de Vigencia en otros tipos de elementos, por lo que debes verificar el universo antes de agrupar.

7. `Base de Datos 28` fue identificada preliminarmente como:

- Object_ID = 102
- Object_Type = 'Class'
- Stereotype = 'ArchiMate_DataObject'

8. En t_connector:

- Start_Object_ID parece representar Source/origen;
- End_Object_ID parece representar Target/destino;
- los conectores relevantes parecen usar Connector_Type = 'Dependency';
- Direction observada: 'Source -> Destination'.

9. Se encontraron preliminarmente 8 dependencias cuyo destino es Base de Datos 28, pero solo 5 tienen como origen una aplicación.

Las otras 3 parten de servidores/nodos y NO satisfacen el interrogante de la consigna.

10. Las cinco aplicaciones dependientes encontradas preliminarmente son:

- Aplicación 5
- Aplicación 12
- Aplicación 22
- Aplicación 25
- Aplicación 27

Verifícalas de forma independiente.

OBJETIVOS DE EXPLORE:

- confirmar o refutar cada hallazgo preliminar;
- documentar estructura del QEA y tablas relevantes;
- demostrar el criterio exacto para "Aplicación";

- demostrar almacenamiento de Categoría;
 - demostrar almacenamiento de Vigencia;
 - revisar NULL, vacío, N/A y tagged values duplicados;
 - demostrar identidad inequívoca de Base de Datos 28;
 - demostrar la dirección source -> target de t_connector;
 - identificar Connector_Type y cualquier otro filtro necesario;
 - distinguir explícitamente dependencias entrantes totales de dependencias cuyo origen es aplicación;
 - evaluar compatibilidad del SQL SQLite con el SQL Search de Enterprise Architect;
 - consultar la documentación InsideEA/Object Model cuando corresponda;
 - considerar desde diseño futuro que las queries deberán ser reutilizables en EA y potencialmente adaptables a Prolaborate.
- Detente para aprobación humana.

QRY-003

P-PROPOSAL — Proposal

Ejecuta únicamente la fase `sdd-propose` para el change `ea-governance-queries-exercise-2`.

Usa como fuentes:

- `exploration.md` aprobado;
- la consigna oficial del Ejercicio 2;
- los hallazgos confirmados del repositorio QEA.

Objetivo:

definir formalmente el alcance, estrategia, restricciones y criterios de validación del Ejercicio 2, sin implementar todavía las consultas SQL.

La propuesta debe contemplar:

- solución exclusivamente SELECT/read-only;
- Enterprise Architect SQL Search como superficie primaria de ejecución y validación;
- SQLite `mode=ro` solo como diagnóstico secundario;
- uso de `Categoría` y `Vigencia` como tagged values;
- identificación de aplicaciones mediante `Class + ArchiMate\_ApplicationComponent`;
- análisis de impacto hacia `Base de Datos 28` respetando source → target;
- resultados derivados del repositorio, nunca hardcodeados;
- evidencia real en EA;
- Prolaborate únicamente como follow-on opcional después de validar los tres interrogantes obligatorios;
- preservación total de `Addino/`, Exercise 1 y el QEA.

Define:

- alcance incluido;
- exclusiones;
- estrategia recomendada;
- criterios de éxito;
- riesgos relevantes;
- decisiones que deberán resolverse en Spec o Design.

No escribas SQL final.

No modifiques el QEA.

No ejecutes Spec, Design, Tasks, Apply, Verify ni Archive.

No hagas commit.

Al finalizar, crea/actualiza `proposal.md`, resume las decisiones principales y detente para revisión humana.

QRY-004

Explore y Proposal de `ea-governance-queries-exercise-2` han sido revisados y APROBADOS por el operador. Continúa ÚNICAMENTE con la fase `sdd-spec` usando Kimi K2.7 Code.

NO ejecute Design, Tasks, Apply, Verify ni Archive.

CHANGE: ea-governance-queries-exercise-2

ROOT: C:\Proagile\Addin

ÁREA DE ENTREGA: C:\Proagile\Addin\Exercise_2_queries

QEA LOCAL: C:\Proagile\Repositorio Pasantias.qea

Toma como fuentes normativas de esta fase:

- `openspec/changes/ea-governance-queries-exercise-2/exploration.md`

- `openspec/changes/ea-governance-queries-exercise-2/proposal.md`

- la consigna disponible bajo `Exercise\_2\_queries/docs/challenge/`

Explore y Proposal ya fueron aprobados. NO redescubras el repositorio ni reabras decisiones congeladas salvo que detectes una contradicción objetiva; si la detectas, repórtala y detente antes de cambiar el alcance.

IMPORTANTE: NO resumas, acortes ni comprimas artificialmente el contenido de la Spec para ajustarlo a un límite de palabras. Conserva todo el detalle funcional, técnico y de aceptación necesario.

FORMATO NATIVO OBLIGATORIO:

La Spec MUST usar encabezados reconocibles por el parser nativo:

`### Requirement:` o `### REQ-n:`

Cada escenario MUST usar:

`#### Scenario:`

Los escenarios deben expresarse con Given / When / Then y usar RFC 2119 (MUST, SHOULD, MAY, MUST NOT) donde corresponda.

OBJETIVO:

Traducir Proposal en requisitos verificables, sin diseñar todavía la SQL final ni decidir prematuramente el número exacto de sentencias.

La Spec debe cubrir como mínimo:

1. UNIVERSO DE APLICACIÓN

El sistema MUST delimitar aplicaciones mediante el criterio defensivo:

`Object\_Type='Class' AND Stereotype='ArchiMate\_ApplicationComponent'`.

El QEA suministrado contiene 30 aplicaciones como baseline de validación, no como constante hardcodeable.

2. Q1 — CATEGORIA ORO

La solución MUST:

- usar el tagged value `Categoria`;
- obtenerlo desde `t\_objectproperties` relacionado con el elemento por `Object\_ID`;
- restringir el universo a aplicaciones;
- responder cantidad total y listado de aplicaciones cuyo valor sea `ORO`;
- identificar/documentar explícitamente el tagged value utilizado;
- derivar los resultados desde el repositorio, nunca hardcodearlos.

Baseline del QEA suministrado:

Total = 8

Aplicación 1, Aplicación 2, Aplicación 3, Aplicación 4, Aplicación 6, Aplicación 8, Aplicación 20, Aplicación 29

3. Q2 — VIGENCIA

La solución MUST:

- usar el tagged value `Vigencia` desde `t\_objectproperties`;
- restringir el universo a aplicaciones;
- MUST NOT utilizar `t\_object.Status` para representar vigencia;
- producir cantidades agrupadas para `Vigente` y `Deprecado`;
- contemplar valores faltantes/no informados mediante validación/documentación sin crear buckets artificiales inexistentes.

Baseline del QEA suministrado:

Vigente = 16

Deprecado = 14

NULL = 0

empty = 0

N/A = 0

Los ceros de missing/unreported son evidencia de validación y documentación, NO filas artificiales obligatorias del resultado agrupado.

4. Q3 — IMPACTO BASE DE DATOS 28

La solución MUST:

- considerar `Dependency`;
- tratar `Start\_Object\_ID` como source/origen;
- tratar `End\_Object\_ID` como target/destino;
- exigir que el source cumpla el criterio de aplicación;
- identificar semánticamente el target mediante:

Name='Base de Datos 28'

Object_Type='Class'

Stereotype='ArchiMate_DataObject'

- MUST NOT depender de Object_ID=102 como identificador funcional principal;
- MAY usar Object_ID=102 únicamente como baseline/evidencia diagnóstica;
- MUST NOT requerir `Direction` como filtro si Start/End ya expresan la orientación;
- devolver cantidad total y listado de aplicaciones afectadas;
- excluir Nodes/servidores del resultado obligatorio.

Baseline:

Total = 5

Aplicación 5

Aplicación 12

Aplicación 22

Aplicación 25

Aplicación 27

Servidor 2, Servidor 5 y Servidor 19 MUST quedar fuera del resultado obligatorio y SHOULD utilizarse como negative-control evidence del filtro de aplicación.

5. QUERY SET

La solución MUST consistir exclusivamente en SELECT/read-only SQL.

La Spec MUST NOT fijar todavía una cantidad exacta de sentencias.

Una o más sentencias MAY resolver cada interrogante cuando Design determine que mejora claridad, compatibilidad, reutilización o trazabilidad.

Cada sentencia final MUST quedar identificada con el interrogante que responde.

6. EA SQL SEARCH

EA SQL Search MUST ser la superficie primaria de ejecución y validación.

Las queries finales MUST validarse realmente en EA antes de considerarse aceptadas.

SQLite `mode=ro` MAY utilizarse como validación diagnóstica secundaria pero MUST NOT sustituir evidencia EA.

7. TRAZABILIDAD

Resultados que representan elementos individuales SHOULD devolver `CLASSGUID` y `CLASSTYPE` cuando corresponda para navegación/trazabilidad en EA.

Queries puramente agregadas MUST NOT inventar un elemento representativo solo para proporcionar esos alias.

8. EVIDENCIA

Debe existir evidencia real de ejecución EA mediante screenshots o exportación de resultados.

NO inventar evidencia.

Debe poder relacionarse cada interrogante con:

pregunta -> SQL -> resultado -> elemento/modelo EA cuando corresponda.

9. DOCUMENTACIÓN

La solución MUST documentar:

- tablas utilizadas;
- campos relevantes;
- joins;
- filtros;
- supuestos;
- criterio de aplicación;
- tagged value utilizado;
- interpretación de los resultados;
- validaciones relevantes.

Las queries MUST ser reutilizables y re-ejecutables y SHOULD incluir comentarios donde aporten claridad.

10. READ-ONLY / SAFETY

El QEA MUST permanecer read-only.

MUST NOT ejecutarse UPDATE, INSERT, DELETE, ALTER, DROP ni operaciones mutantes.

Addino/E1 MUST permanecer sin modificaciones.

13. PROLABORATE

Prolaborate es OPTIONAL FOLLOW-ON.

Solo después de validar los tres interrogantes obligatorios en EA MAY:

- reutilizarse al menos una query validada;
- crearse donut/bar/dashboard;
- documentarse configuración y pregunta de negocio;
- capturarse evidencia visual.

La imposibilidad de completar este punto extra MUST NOT invalidar la aceptación obligatoria.

14. CRITERIOS DE CALIDAD

Los Requirements/Scenarios deben hacer verificables:

- correctitud;
- claridad técnica;
- reutilización;
- trazabilidad;
- read-only;
- evidencia real;
- cumplimiento de entregables.

NO escribas SQL final.

NO decidas todavía nombres finales concretos.

NO modifiques `Exercise\_2\_queries/`.

NO modifiques el QEA.

NO modifiques Addino/E1.

NO hagas commit.

NO avances a Design.

Al finalizar:

- indica el/los archivo(s) de Spec creado(s);
- enumera los Requirements y cantidad de Scenarios;
- confirma que el parser nativo debería reconocer los headings `### Requirement:` / `### REQ-n:` y `##### Scenario:`;
- ejecuta `git diff --check`;
- muestra `git status --short`;
- detente para revisión humana.

QRY-005

Explore, Proposal y las tres Specs de `ea-governance-queries-exercise-2` han sido revisados y APROBADOS por el operador.

Continúa ÚNICAMENTE con la fase `sdd-design` usando GPT-5.6 Sol.

NO ejecute Tasks, Apply, Verify ni Archive.

CHANGE: ea-governance-queries-exercise-2

ROOT: C:\Proagile\Addin

ÁREA DE ENTREGA: C:\Proagile\Addin\Exercise_2_queries

QEA LOCAL DE TRABAJO: C:\Proagile\Repositorio Pasantias.qea

FUENTES NORMATIVAS:

- `openspec/changes/ea-governance-queries-exercise-2/exploration.md`
- `openspec/changes/ea-governance-queries-exercise-2/proposal.md`
- las tres Specs aprobadas bajo
`openspec/changes/ea-governance-queries-exercise-2/specs/`
- la consigna bajo `Exercise\_2\_queries/docs/challenge/`
- documentación local de InsideEA / Object Model cuando sea necesaria para decisiones de compatibilidad EA
- guía de Prolaborate únicamente para el optional follow-on.

IMPORTANTE: NO resumas, acortes ni comprimas artificialmente `design.md` para ajustarlo a un límite de palabras. Documenta con el nivel de detalle técnico necesario para que Tasks y Apply puedan implementarlo sin reinterpretar decisiones.

NOTA DE CONTEO:

El resumen anterior del agente distribuyó incorrectamente Requirements/Scenarios por archivo. NO modifiques las Specs por este motivo.

El conteo real de headings es:

- Category: 9 Requirements / 14 Scenarios
- Lifecycle: 7 Requirements / 11 Scenarios
- Impact: 9 Requirements / 13 Scenarios
- TOTAL: 25 Requirements / 38 Scenarios.

Las Specs están aprobadas. Si detectas una contradicción funcional nueva y objetiva, repórtala y detente; no reabras decisiones arbitrariamente.

OBJETIVO DE DESIGN:

Definir CÓMO se implementará Exercise 2, incluyendo decisiones SQL, estructura de entregables, compatibilidad EA, validación y evidencia, sin crear todavía los archivos finales de implementación. A diferencia de Spec, Design ahora DEBE tomar una decisión explícita y razonada sobre la descomposición final del query set.

EVALÚA Y DECIDE:

1. QUERY SET / SENTENCIAS

Decide cuántas sentencias SELECT finales habrá y cómo se distribuyen entre Q1, Q2 y Q3.

La decisión debe priorizar:

- claridad para el evaluador;
- compatibilidad con EA SQL Search;
- trazabilidad;
- reutilización;
- facilidad de capturar evidencia;
- cumplimiento de cantidad + listado donde corresponda.

Puede elegirse una o más sentencias por interrogante. Documenta la decisión y su rationale.

NO implementes todavía el SQL final.

2. Q1 — CATEGORIA ORO

Diseña la estrategia SQL para derivar:

- total de aplicaciones ORO;
- listado de aplicaciones ORO;
- aplicaciones lógicamente únicas.

Debe usar:

- t_object;
- t_objectproperties;
- join por Object_ID;
- Property='Categoria';
- Value='ORO';
- criterio defensivo de aplicación.

Baseline supplied-QEA:

8 aplicaciones:

1, 2, 3, 4, 6, 8, 20, 29.

Decide cómo asegurar unicidad sin hardcodear resultados.

Para resultados individuales, toma una decisión explícita sobre implementar el SHOULD de CLASSGUID/CLASSTYPE para navegación EA y documenta rationale.

3. Q2 — VIGENCIA

Diseña una consulta agrupada que derive:

- Vigente=16;
- Deprecado=14;

sobre aplicaciones únicas.

Debe usar tagged value Vigencia y MUST NOT usar t_object.Status.

Debe documentarse:

NULL=0

empty=0

N/A=0

en el supplied QEA, sin generar buckets artificiales.

No inventes CLASSGUID/CLASSTYPE para filas agregadas.

No agregues un listado individual de Q2 salvo que exista una razón clara y justificada; no es obligatorio.

4. Q3 — IMPACTO DB28

Diseña la estrategia SQL para derivar:

- total de aplicaciones afectadas;
- listado de aplicaciones afectadas;
- aplicaciones lógicamente únicas.

Semántica obligatoria:

source t_object application

→ t_connector Dependency

→ target t_object Base de Datos 28.

Source:

Object_Type='Class'

Stereotype='ArchiMate_ApplicationComponent'

Target:

Name='Base de Datos 28'

Object_Type='Class'

Stereotype='ArchiMate_DataObject'

Object_ID=102 es solamente validation oracle.

NO hardcodearlo como mecanismo principal.

Start_Object_ID = Source.

End_Object_ID = Target.

NO añadir Direction como filtro obligatorio.

Baseline:

5 aplicaciones:

5, 12, 22, 25, 27.

Servidor 2, 5 y 19 quedan fuera del resultado obligatorio. Decide cómo utilizar, si resulta útil, ese negative control en validación/documentación.

5. UNICIDAD

Design DEBE elegir una estrategia SQL concreta para garantizar aplicaciones únicas en Q1/Q2/Q3. Evalúa la alternativa más compatible y legible para EA SQL Search, por ejemplo DISTINCT, COUNT(DISTINCT ...), agrupación u otra forma apropiada.

Justifica la decisión por compatibilidad, legibilidad y semántica.

No uses técnicas complejas si una solución SQL simple y portable satisface los requisitos.

6. COMPATIBILIDAD EA SQL SEARCH

Diseña pensando primero en Enterprise Architect SQL Search, no solo SQLite.

Prioriza SQL simple:

- SELECT;
- JOIN;
- WHERE;

- GROUP BY;
- ORDER BY;
- DISTINCT/agregación cuando se justifique.

Evita CTEs, window functions, extensiones específicas o construcciones innecesarias salvo evidencia de necesidad/compatibilidad.

Documenta qué partes necesitarán validación real dentro de EA.

Las queries que devuelvan elementos individuales deberían ser navegables mediante CLASSGUID/CLASSTYPE si Design adopta ese SHOULD.

Las agregadas no deben inventar esos alias.

7. ESTRUCTURA DE ENTREGABLES

Ahora Design PUEDE elegir los nombres finales concretos.

Todos los archivos submission-facing MUST incluir `Pino`.

Diseña la estructura final dentro de `Exercise\_2\_queries/`, cubriendo como mínimo:

- archivo SQL;
- explicación técnica;
- evidencia EA;
- AI Usage Log;
- documentación/área Prolaborate opcional;
- README/índice si aporta claridad.

NO modifiques ni elimines `docs/challenge/`.

Define el propósito de cada archivo y cómo se relacionan.

8. EVIDENCIA Y HUMAN GATES

Diseña un flujo de validación incremental.

La ejecución real en EA requiere intervención humana y no puede ser simulada.

Define cómo Apply/Tasks deberían trabajar de manera incremental, preferentemente:

Q1 implementación → SQLite secundaria → EA real → evidencia

Q2 implementación → SQLite secundaria → EA real → evidencia

Q3 implementación → SQLite secundaria → EA real → evidencia

documentación consolidada

Prolaborate opcional

Verify.

Define qué evidencia se debe capturar por interrogante y cómo demostrar:

pregunta → SQL → resultado → modelo/elemento EA.

NO inventes screenshots ni resultados EA.

9. SQLITE SECUNDARIO

Diseña SQLite `mode=ro` solo como oráculo/validación independiente.

Debe permitir comparar contra:

Q1: 8 + lista

Q2: 16/14 + missing 0

Q3: 5 + lista, con Nodes como control si se utiliza.

Nunca sustituye la validación EA.

11. PROLABORATE OPTIONAL FOLLOW-ON

Diseña solamente la estrategia opcional posterior a la validación EA.

Consulta la guía adjunta/local.

La guía permite construir un Donut mediante SQL Queries entrando en Query Configuration, introduciendo y ejecutando la query antes de configurar el chart.

Evalúa cuál de las queries obligatorias sería el candidato más claro para visualización y documenta la recomendación con rationale, pero NO la implementes todavía.

No introduzcas aliases específicos como `seriesproperty` salvo que la funcionalidad elegida realmente los necesite; la guía lo relaciona específicamente con colores individuales definidos mediante Color Palette Configuration.

Prolaborate sigue sin bloquear la entrega obligatoria.

12. READ-ONLY Y LÍMITES

QEA estrictamente read-only.

No UPDATE/INSERT/DELETE/ALTER/DROP/CREATE.

No modificar Addino/E1.

No modificar RDD/review mode.

No introducir C#.

No mover ni renombrar el QEA.

13. TRAZABILIDAD DE DISEÑO

Incluye una matriz o sección que muestre cómo las decisiones de Design satisfacen las tres capability Specs y los requisitos transversales.

Distingue claramente:

- requirement;
- design decision;
- validation/evidence mechanism.

14. RIESGOS / DECISIONES

Documenta architecture/design decisions con rationale, incluyendo al menos:

- statement decomposition;
- uniqueness strategy;
- EA alias strategy;
- target identification strategy;
- EA vs SQLite validation;
- evidence organization;
- optional Prolaborate candidate;
- human validation gates.

NO escribas los archivos SQL finales.

NO modifiques `Exercise\_2\_queries`.

NO ejecutes queries de implementación como producto final.

Las consultas diagnósticas read-only solo están permitidas si son necesarias para resolver una decisión de Design.

NO modifiques Specs/Proposal/Explore.

NO hagas commit.

NO ejecutes Tasks.

Al finalizar:

- indica `design.md` creado/modificado;
- resume las decisiones de arquitectura tomadas;
- informa el número final de sentencias SELECT elegido y su distribución Q1/Q2/Q3;
- informa la estrategia de unicidad;
- informa los archivos finales propuestos;
- informa la estrategia de evidencia y human gates;
- confirma que no implementaste nada;
- ejecuta `git diff --check`;
- muestra `git status --short`;
- detente para revisión humana.

QRY-006

Explore, Proposal, Specs y Design de `ea-governance-queries-exercise-2` han sido revisados y APROBADOS por el operador.

Continúa ÚNICAMENTE con la fase `sdd-tasks` usando Qwen 3.7 Plus.

NO ejecutes Apply, Verify ni Archive.

CHANGE: ea-governance-queries-exercise-2

ROOT: C:\Proagile\Addin

ÁREA DE ENTREGA: C:\Proagile\Addin\Exercise_2_queries

QEA LOCAL DE TRABAJO: C:\Proagile\Repositorio Pasantias.qea

FUENTES NORMATIVAS:

- exploration.md aprobado
- proposal.md aprobado
- las tres Specs aprobadas
- design.md aprobado
- consigna bajo `Exercise\_2\_queries/docs/challenge/`

Design es ahora la autoridad sobre decisiones de implementación.

IMPORTANTE: NO resumas, acortes ni comprimas artificialmente `tasks.md` para ajustarlo a una cantidad de palabras o tareas. El objetivo es producir un plan ejecutable, incremental y verificable, no minimizar el número de tasks.

NO reabras decisiones de Design salvo contradicción objetiva. Si detectas una contradicción objetiva que impide generar Tasks, repórtala y detente.

DECISIONES CERRADAS:

- exactamente cinco SELECTs finales: Q1-C, Q1-L, Q2-G, Q3-C, Q3-L
- distribución: Q1 = 2, Q2 = 1, Q3 = 2
- un único archivo `.sql` de entrega, NO ejecutado como batch;
- cada SELECT se copia/crea/ejecuta individualmente en EA SQL Search;
- texto ejecutable EA comienza en SELECT;
- Q1-C = COUNT(DISTINCT app.Object_ID);
- Q1-L = SELECT DISTINCT con columnas estables por aplicación;
- Q2-G = grouped Vigencia + COUNT(DISTINCT app.Object_ID);
- Q3-C = COUNT(DISTINCT src.Object_ID);
- Q3-L = SELECT DISTINCT con columnas estables por aplicación source;
- Q1-L/Q3-L incluyen CLASSGUID/CLASSTYPE;
- agregados no incluyen alias representativos;
- no PropertyID ni Connector_ID en listados finales cuando rompan deduplicación;
- Q3 identifica target semánticamente, nunca por Object_ID=102 como filtro;
- no Direction;
- EA SQL Search = autoridad primaria;
- SQLite mode=ro = diagnóstico secundario;
- Prolaborate = opcional, solo después de Q1/Q2/Q3 EA;
- Q2-G es candidato Prolaborate;
- toda evidencia submission-facing lleva Pino;
- Addino/E1 y QEA protegidos.

ESTRUCTURA OBLIGATORIA DE TASKS:

Agrupar las tareas por fases/work units y mantener cada task realizable en una sesión.

El plan DEBE separar claramente trabajo automatizable del agente y HUMAN GATES que requieren al operador.

No marques una tarea humana como realizable por el agente.

FASE 1 — SCAFFOLDING Y TRAZABILIDAD

Planifica la creación, durante Apply, de la estructura aprobada:

`Exercise\_2\_queries/queries/Pino\_Exercise\_2\_EA\_Governance\_Queries.sql`

`Exercise\_2\_queries/docs/Pino\_Exercise\_2\_Technical\_Explainer.md`

`Exercise\_2\_queries/docs/Pino\_Exercise\_2\_AI\_Usage\_Log.md`

`Exercise\_2\_queries/evidence/Pino\_Exercise\_2\_Evidence\_Index.md`

carpetas: `evidence/ea/`, `evidence/sqlite/`, `evidence/prolaborate/`, `prolaborate/`

Preservar `docs/challenge/`.

El Evidence Index puede contener estados PENDING antes de ejecución real, pero MUST NOT presentar placeholders como evidencia ejecutada.

No crear PNG ficticios ni archivos vacíos que pretendan ser screenshots.

FASE 2 — Q1 CATEGORIA ORO

Planifica tasks para:

1. Implementar Q1-C.
2. Implementar Q1-L.
3. Revisar estáticamente filtros, alias y unicidad.
4. Ejecutar diagnóstico SQLite estrictamente mode=ro.
5. Verificar oracle:

count = 8

list = Aplicación 1,2,3,4,6,8,20,29.

6. Actualizar documentación secundaria con resultados reales del diagnóstico.

7. DETENERSE EN HUMAN GATE Q1.

HUMAN GATE Q1:

El operador ejecutará Q1-C y Q1-L individualmente en EA SQL Search, idealmente con nombres:

`Q1-C — Categoría ORO — Count`

`Q1-L — Categoría ORO — List`

Debe comprobar:

- count 8;
- lista exacta;
- Categoría visible;
- navegación desde al menos una fila de Q1-L;
- evidencia real screenshot/export.

Apply MUST detenerse antes de continuar a Q2 si todavía no existe confirmación humana de este gate.

Planifica después una task para incorporar/indexar la evidencia REAL proporcionada por el operador.

FASE 3 — Q2 VIGENCIA

Solo después de HUMAN GATE Q1 aprobado.

Planifica:

1. Implementar Q2-G.
2. Verificar que usa tag Vigencia y nunca t_object.Status.
3. Verificar aplicación scope.
4. Verificar COUNT(DISTINCT app.Object_ID).
5. Diagnóstico SQLite mode=ro.
6. Confirmar:

Vigente=16

Deprecado=14

NULL=0

empty=0

N/A=0.

7. Comprobar que no existen Vigencias conflictivas por aplicación.

8. Si aparece conflicto, registrar anomalía y detener Q2 sin inventar precedencia.

9. DETENERSE EN HUMAN GATE Q2.

HUMAN GATE Q2:

El operador ejecuta Q2-G individualmente en EA y comprueba:

- 16/14;

- comparación con tagged values visibles;

- ausencia de buckets artificiales;

- evidencia real.

Después, task para incorporar/indexar evidencia real.

FASE 4 — Q3 IMPACTO DB28

Solo después de HUMAN GATE Q2 aprobado.

Planifica:

1. Diagnóstico read-only de cardinalidad del target semántico.

2. Confirmar exactamente un target:

Name='Base de Datos 28'

Object_Type='Class'

Stereotype='ArchiMate_DataObject'.

3. Object_ID=102 solo se registra como oracle, nunca filtro.

4. Si cardinalidad != 1, registrar anomalía y detener Q3.

5. Implementar Q3-C.

6. Implementar Q3-L.

7. Verificar Start_Object_ID=source y End_Object_ID=target.

8. Verificar Connector_Type='Dependency'.

9. Verificar ausencia de Direction predicate.

10. Verificar source application scope.

11. Diagnóstico SQLite mode=ro.

12. Confirmar:

count = 5

Aplicación 5,12,22,25,27.

13. Servidor 2,5,19 pueden documentarse como negative-control evidence pero MUST NOT aparecer en resultado obligatorio.

14. DETENERSE EN HUMAN GATE Q3.

HUMAN GATE Q3:

El operador ejecuta Q3-C/Q3-L individualmente en EA.

Debe comprobar:

- count 5;

- lista exacta;

- Nodes excluidos;

- source/target Dependency visible en el modelo;

- navegación desde al menos una fila Q3-L;

- evidencia real.

Después, task para incorporar/indexar evidencia real.

FASE 5 — COMPATIBILIDAD EA

Incluye tasks explícitas para manejar fallos reales de dialecto.

En particular:

- probar COUNT(DISTINCT app.Object_ID);
- probar COUNT(DISTINCT src.Object_ID);
- probar alias CLASSGUID/CLASSTYPE;
- probar acentos;
- probar ORDER BY;
- comprobar que cada bloque ejecutado comienza en SELECT.

Si EA rechaza COUNT(DISTINCT ...):

- NO cambiar semántica;
- documentar evidencia del fallo;
- implementar únicamente un fallback SQL básico read-only compatible;
- volver a pasar SQLite diagnostic + HUMAN EA gate correspondiente.

No anticipar fallback si EA acepta la forma primaria.

FASE 6 — DOCUMENTACIÓN FINAL OBLIGATORIA

Solo después de Q1/Q2/Q3 human gates aprobados.

Planifica completar

Technical Explainer:

- pregunta de negocio;
- statement ID;
- tablas;
- campos;
- joins;
- filtros;
- criterio de aplicación;
- tagged values;
- estrategia de unicidad;
- target DB28;
- Direction decision;
- EA vs SQLite;
- oracles;
- supuestos/anomalías;
- instrucciones reproducibles de ejecución individual en EA.

Evidence Index:

- question;
- Statement ID;
- SQL location;
- EA Search display name;
- expected oracle;
- actual EA result;
- model comparison;
- evidence filename;
- operator/date;
- pass/fail.

AI Usage Log:

>=5 interacciones significativas REALES.

Cada entrada MUST tener ocho campos separados:

1 ID

2 objetivo

3 herramienta

4 modelo

5 estrategia/prompt

6 decisión tomada

7 evidencia relacionada

8 resultado

Puede documentar retrospectivamente interacciones genuinas ya realizadas si todos los campos se recuperan fielmente.

No fabricar prompts, modelos, decisiones ni evidencia.

FASE 7 — PROLABORATE OPTIONAL FOLLOW-ON

Marcar todas estas tasks como opcionales y solo habilitadas después de Q1/Q2/Q3 aprobados en EA.

Plan:

- partir de Q2-G EA-validada;
- Create Chart Widget;
- SQL Queries;
- Skip to Query;
- Query Configuration;
- View Sample cuando sea útil;
- Execute;
- si funciona directamente, reutilizar Q2-G;
- si Prolaborate exige shape/alias de presentación, permitir adaptación mínima documentada que preserve exactamente semántica/resultados;
- EA Q2-G sigue siendo autoridad;
- documentar business question y configuración;
- capturar evidencia visual real;
- toda evidencia lleva Pino.

No incluir `seriesproperty` salvo necesidad real demostrada por Color Palette Configuration.

La ausencia de Prolaborate MUST NOT impedir completar el alcance obligatorio.

FASE 8 — PRE-VERIFY

Planifica una auditoría final antes de `sdd-verify`:

- exactamente 5 SELECTs finales;
- todos SELECT/read-only;
- QEA sin modificación;
- Addino/E1 sin modificación;
- naming Pino;
- query → result → model → evidence trazable;
- oracles correctos;
- evidencia real por cada statement;
- AI log completo;
- links/rutas relativas válidas;
- ningún placeholder presentado como prueba;
- `git diff --check`;
- `git status --short`.

IMPORTANTE PARA APPLY:

Tasks debe dejar explícitos los STOP POINTS.

No debe diseñar un futuro `sdd-apply` que implemente Q1/Q2/Q3 de una sola vez ignorando validaciones humanas.